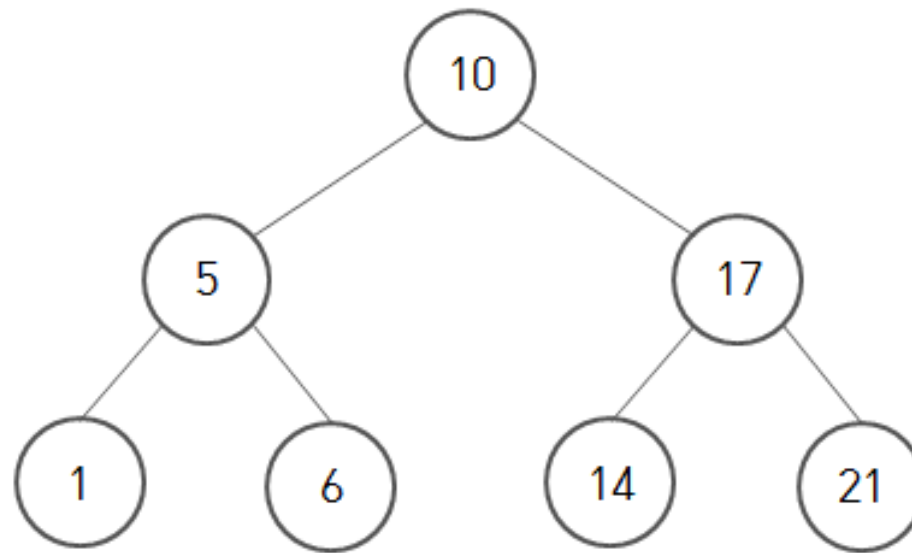


체스 Agent 제작 2주차

트리 탐색 기반 에이전트 구현

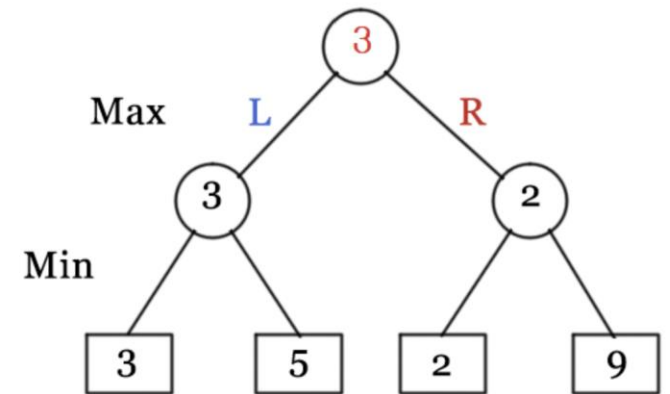
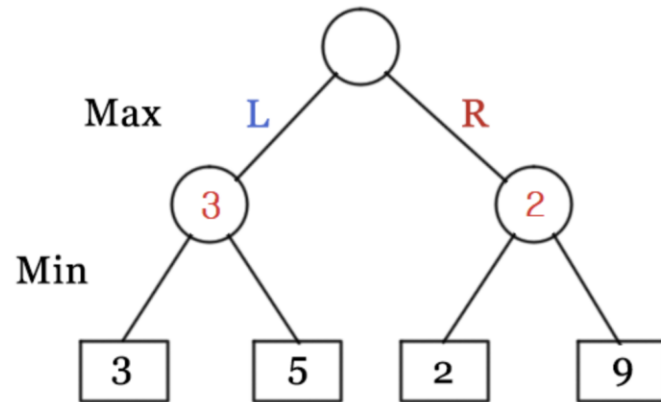
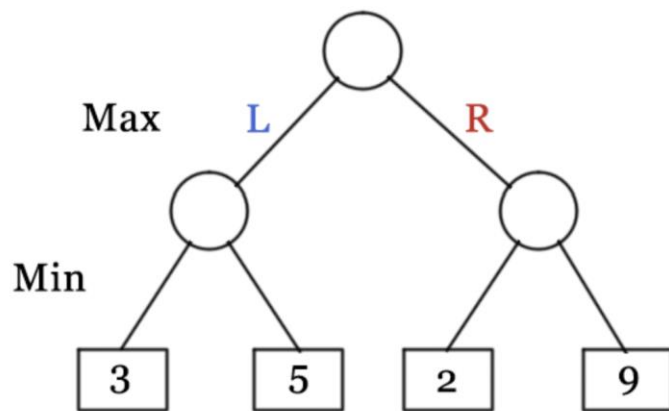
1. 트리 탐색이란?

- 최적의 결과 하나를 찾기 위해 수많은 가능 경로를 탐색하는 과정
- MCTS, 미니맥스, Alpha-beta pruning 등의 알고리즘 존재
- 예시 Agent는 Alpha-beta pruning 알고리즘을 활용하여 제작



2. Alpha-beta pruning

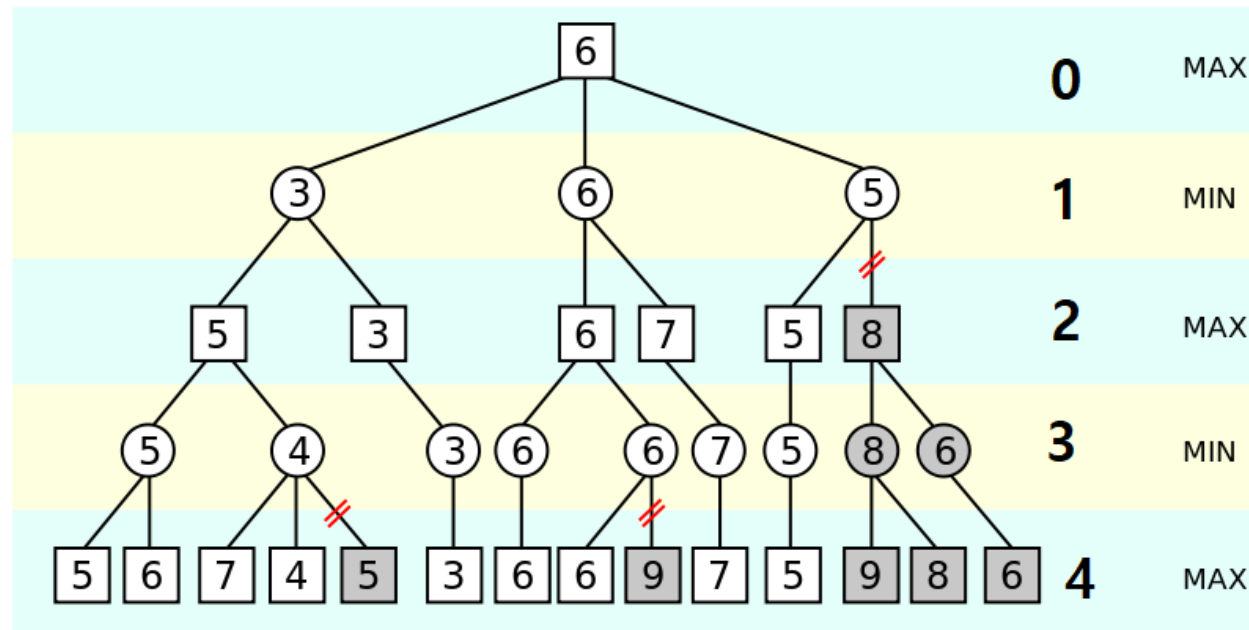
- Minimax algorithm
 - Minimax algorithm이란: 본인과 상대방 모두 최고의 수를 둔다고 가정했을 때의 탐색 방법
 - 예시: <그림 1>, 2수 깊이 탐색
 - 첫 수는 본인의 수, 두 번째 수는 상대방의 수
 - 두 번째 수의 경우 상대방의 수이므로 점수가 작은 점수를 선택, 첫 수의 경우 자신의 수이므로 점수가 큰 수를 선택



<그림 1>

2. Alpha-beta pruning

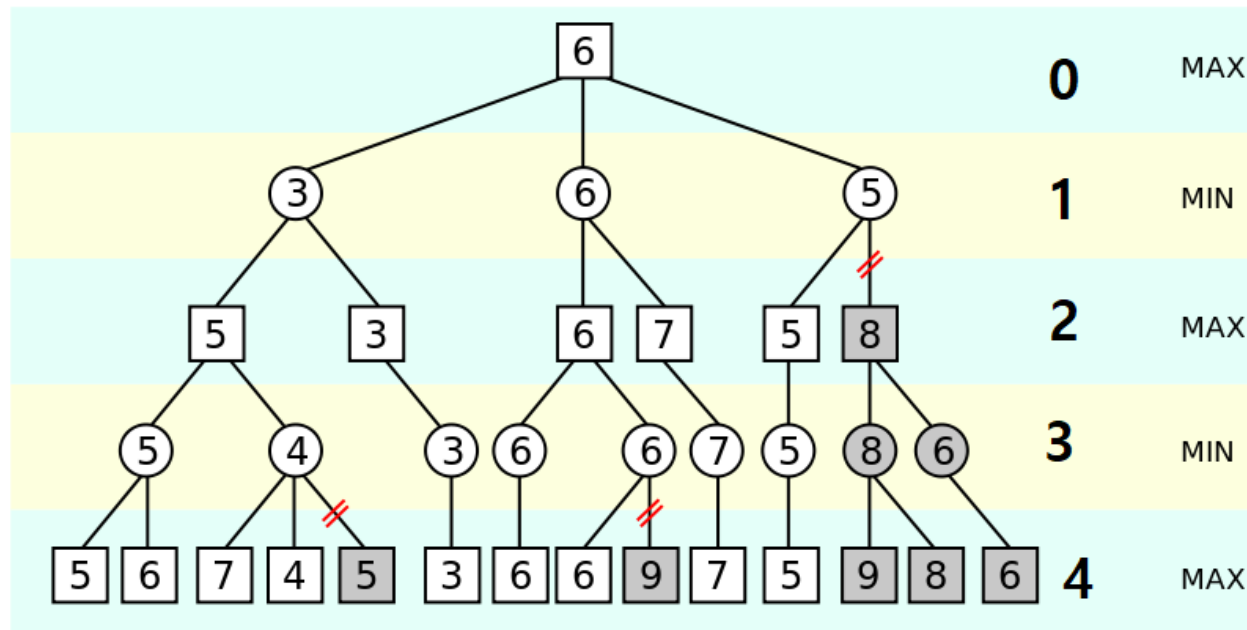
- Minimax algorithm의 문제
 - 트리를 완성하기 위해서는 모든 경우의 수를 조사해야 함
 - 깊이가 깊어질수록 탐색해야 할 수의 숫자가 증가하게 됨
 - Alpha-beta pruning: 트리 탐색 과정에서 조사해야 하는 수의 숫자를 줄임



<그림 2>

2. Alpha-beta pruning

- Alpha-beta pruning 알고리즘
 - 기본 원리: 어떤 수를 탐색하는 도중 상대가 그 수를 선택하지 않을 것이 확실할 경우 그 수 이하의 경우는 탐색하지 않음
 - Alpha: 현재까지 본인이 보장할 수 있는 가장 좋은 점수, Beta: 상대가 허용할 수 있는 가장 낮은 점수



<그림 2>

2. Alpha-beta pruning

- Alpha-beta pruning 알고리즘

- Alpha: 현재까지 본인이 보장할 수 있는 가장 좋은 점수, Beta: 상대가 허용할 수 있는 가장 낮은 점수

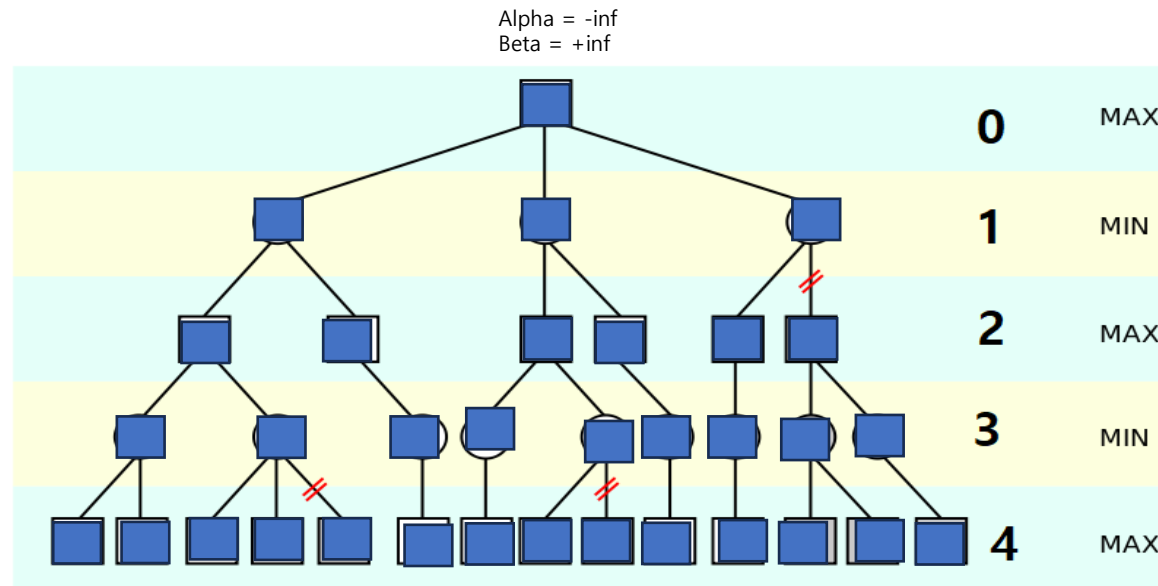
- 기본 규칙 4가지

- 1) 가장 위 node에 $\alpha = -\infty$, $\beta = +\infty$ 를 설정

- 2) 탐색이 진행되면서 MAX 노드는 현재까지 확인한 자식값 중 최댓값을 이용해 α 를 증가시키고, MIN 노드는 현재까지 확인한 자식값 중 최솟값을 이용해 β 를 감소 (이유: Max node는 플레이어 본인이 고르는 수이고, Min node는 상대방이 고르는 수이기 때문)

- 3) 각 노드의 α , β 값은 자식 하나의 탐색이 끝날 때마다 갱신되며, 갱신된 값은 다음 자식 노드로 전달

- 4) 탐색 도중 부모 node에서 $\alpha \geq \beta$ 가 되면 남아 있는 자식 노드는 탐색하지 않고 가지치기(이유는 예시를 통해 설명)



<그림 2>

2. Alpha-beta pruning

- Alpha-beta pruning 알고리즘

- Ex) <그림 2> 예시

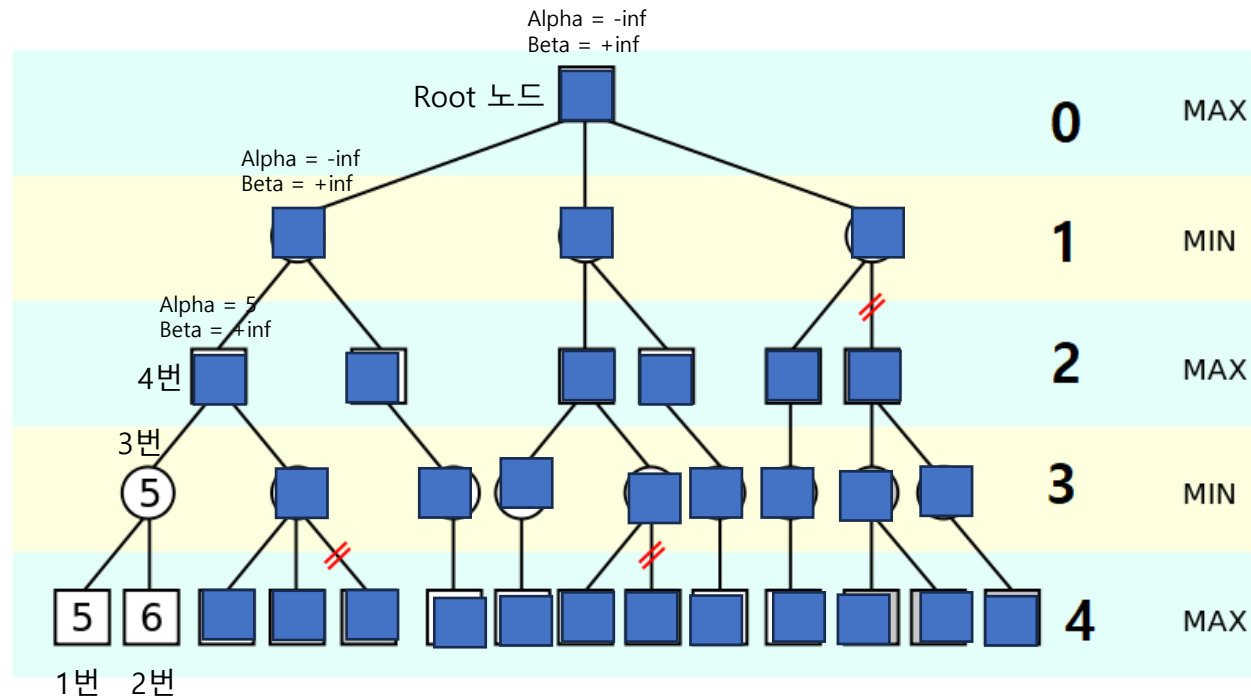
1) $\alpha = -\infty$, $\beta = -\infty$ 상태가 전체 노드에 유지

2) 1,2번 node 탐색 -> 경로에 있는 node들 중 아직 α , β 값 할당이 이루어지지 않은 경우 부모 node의 α , β 값 전달

참고: 1,2번 node는 말단 점수 node이고, 3번 node의 경우 탐색이 완료되어 값이 확정된 node들이므로 α , β 값 할당이 무의미해 넣지 않았음.

1) 4번 node: max node이므로 α 값 업데이트, 현재까지 확보할 수 있는 최대 점수는 5이므로 $\alpha = 5$ 로 업데이트

2) 이외 node들은 자식 node로부터 확실하게 얻을 수 있는 정보가 없으므로 업데이트하지 않음



<그림 2>

2. Alpha-beta pruning

• Alpha-beta pruning 알고리즘

- Ex) <그림 2> 예시

5) 5번, 6번 node 탐색 -> 경로 중 alpha, beta값이 할당되지 않은 node의 경우 부모의 alpha, beta값을 할당하고 그렇지 않은 node의 경우 기존 값 유지(이 경우 8번 node의 alpha, beta값을 4번 node의 값으로 설정)

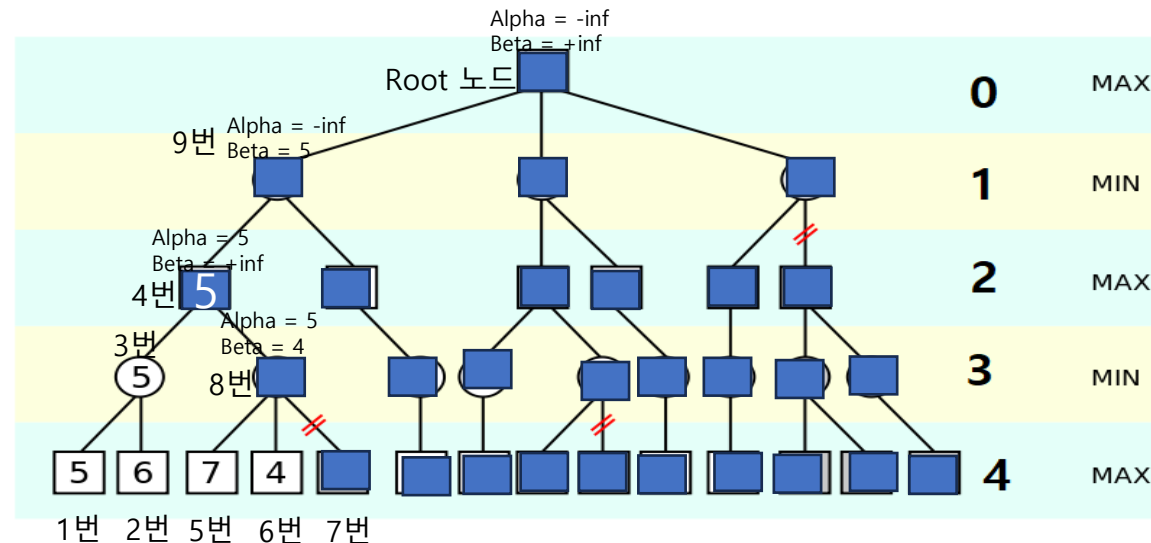
6) 8번 node의 경우 min node이고, 자식 node로부터 현 시점 확실하게 받을 수 있는 최솟값은 4점 -> beta값을 4로 변경

7) 7번 node값은 탐색할 필요가 없음.

→ 이유: 7번 node의 부모인 8번 node의 $\alpha \geq \beta$ 가 되었기 때문. 만약 7번 node의 값이 4 이상이라면 beta값은 변경되지 않을 것임. 7번 node의 값이 4 미만이라면 8번 node에서 beta값은 변경되겠지만, 4번 node에서 8번 node의 값이 무엇이든 간에 4를 넘지 못하므로 3번 node의 5를 선택할 것이기 때문에 4번 node의 alpha값을 변경하는데 영향을 미치지 못함. 따라서 7번 node의 수를 보는 것은 무의미하고, 8번 node의 값도 4로 확정해도 문제가 없음.

8) 4번 node에서 5보다 큰 값을 얻지 못할 것이므로, 4번 node값을 5로 확정 -> 9번 node에서 현 상황 얻을 수 있는 가장 작은 값이 5이므로 $\beta = 5$ 로 변경

9) 더 이상 상위 node값을 업데이트 할 수 없으므로 다른 말단 수 탐색으로 넘어감



<그림 2>

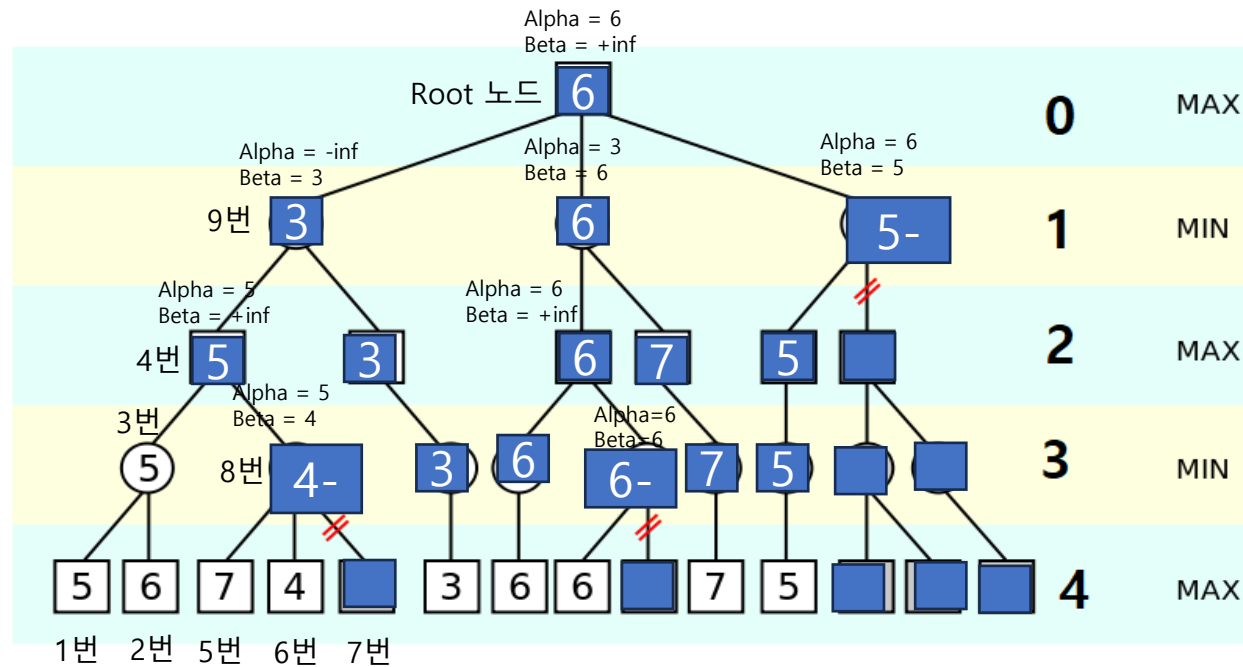
2. Alpha-beta pruning

- Alpha-beta pruning 알고리즘

- Ex) <그림 2> 예시

10) 이전과 같은 과정을 반복하여 트리를 완성(탐색은 말단 node 기준 왼쪽 -> 오른쪽 방향으로 이루어졌다고 가정)/

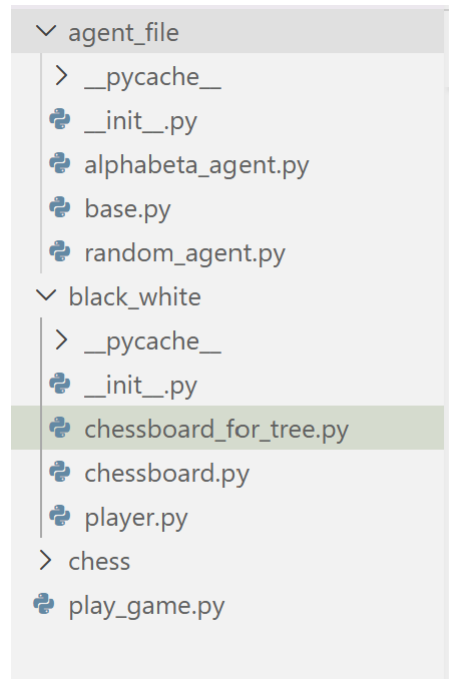
-> 최종적으로 최선의 결과는 6임을 확인할 수 있으며, 현재 트리에서 얻을 수 있는 최대 결과는 6임을 확인할 수 있다.



<그림 2>

3. Alpha beta pruning을 위한 준비 - chessboard_for_tree.py

- 이전 black_white 폴더에서 chessboard_for_tree.py 파일 생성
- 이후 이전에 작성했던 chessboard.py의 내용을 붙여넣음



<그림 3>

3. Alpha beta pruning을 위한 준비 - chessboard_for_tree.py

- 각 기물의 점수 설정
- King 기물 점수는 포함하지 않고, 대신 checkmate 점수를 넣음

```
PIECE_SCORE = {  
    Piece.pawn: 100,  
    Piece.knight: 320,  
    Piece.bishop: 330,  
    Piece.rook: 500,  
    Piece.queen: 900,  
    Piece.king: 0,  
}  
  
CHECKMATE_SCORE = 100000
```

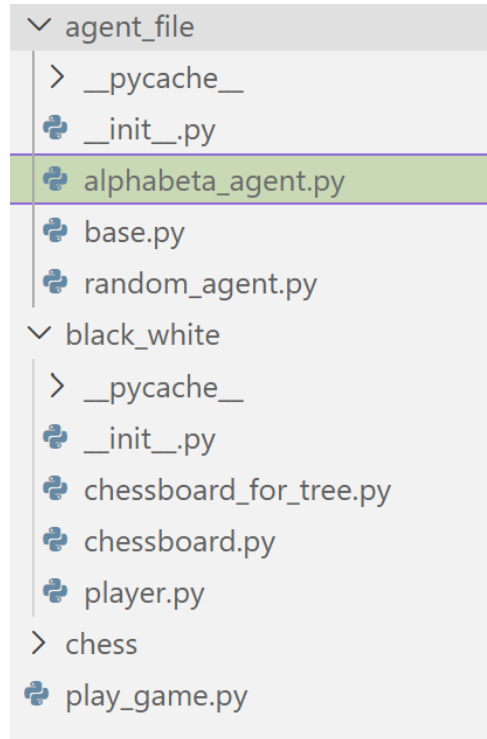
3. Alpha beta pruning을 위한 준비 - chessboard_for_tree.py

- GameState class에 여러 보조 함수를 추가
- apply_move_fast: apply_move에서 legal_moves 체크를 뺀 함수
- score: 수를 둔 후 board 위에 있는 기물들 간의 점수 차이를 계산하는 함수

```
def apply_move_fast(self, move):  
    next_board = self.board.place_stone(move)  
  
    return GameState(  
        board=next_board,  
        next_player=self.next_player.other,  
    )  
  
def score(self, player=None):  
    if player is None:  
        player = self.next_player  
  
    total = 0  
  
    for point, item in self.board.grid.items():  
        piece_player, piece = item  
        value = PIECE_SCORE[piece]  
  
        if piece_player == player:  
            total += value  
        else:  
            total -= value  
  
    return total
```

3. Alpha beta pruning을 위한 준비 -alphabeta_agent.py

- 이전에 만들었던 Agent_file에 alphabeta_agent.py 추가
- 초반 필요한 패키지를 import, 필요한 변수들 추가



<그림 4>

```
import random

from black_white.chessboard_for_tree import *
from agent_file.base import Agent

PIECE_SCORE = {
    Piece.pawn: 100,
    Piece.knight: 320,
    Piece.bishop: 330,
    Piece.rook: 500,
    Piece.queen: 900,
    Piece.king: 0,
}

CHECKMATE_SCORE = 100000
__all__ = ["Alphabeta_bot"]
```

3. Alpha beta pruning을 위한 준비 -alphabeta_agent.py

- Alphabotot(Agent class 상속)
- __init__: depth - 탐색할 수의 깊이, player: 플레이어의 돌 색깔

```
class Alphabotot(Agent):
    def __init__(self, depth=3):
        self.depth = depth
        self.player = None

    def select_move(self, game_state):
        moves = game_state.legal_moves()

        if not moves:
            return None

        if self.player is None:
            self.player = game_state.next_player

        # 루트에서는 내 입장에서 좋은 수를 먼저 탐색
        moves = self.order_moves(game_state, moves, maximizing_player=True)

        best_score = float("-inf")
        best_moves = []

        alpha = float("-inf")
        beta = float("inf")
```

```
        for move in moves:
            next_state = game_state.apply_move_fast(move)

            score = self._alphabeta(
                game_state=next_state,
                depth=self.depth - 1,
                alpha=alpha,
                beta=beta,
                maximizing_player=False,
            )

            if score > best_score:
                best_score = score
                best_moves = [move]
            elif score == best_score:
                best_moves.append(move)

            alpha = max(alpha, best_score)

        return random.choice(best_moves)
```

3. Alpha beta pruning을 위한 준비 -alphabeta_agent.py

- Alphabotot(Agent class 상속)

- select_move

- 1) legal_moves를 호출해 가능한 수를 모집
- 2) Order_moves: 트리 탐색 시 점수가 높은 수부터 정렬하여 alphabeta pruning을 효율적으로 진행하게 하는 함수
- 3) Best_score, best_move, alpha, beta 등 필요한 변수 준비
- 4) 각 move를 적용해 보고, _alphabeta 함수를 통해 해당 수에서 얻을 수 있는 기댓값을 계산 -> 최대 점수를 얻는 move들 중 하나를 random하게 선택함

```
class Alphabotot(Agent):
    def __init__(self, depth=3):
        self.depth = depth
        self.player = None

    def select_move(self, game_state):
        moves = game_state.legal_moves()

        if not moves:
            return None

        if self.player is None:
            self.player = game_state.next_player

        # 루트에서는 내 입장에서 좋은 수를 먼저 탐색
        moves = self.order_moves(game_state, moves, maximizing_player=True)

        best_score = float("-inf")
        best_moves = []

        alpha = float("-inf")
        beta = float("inf")
```

```
        for move in moves:
            next_state = game_state.apply_move_fast(move)

            score = self._alphabeta(
                game_state=next_state,
                depth=self.depth - 1,
                alpha=alpha,
                beta=beta,
                maximizing_player=False,
            )

            if score > best_score:
                best_score = score
                best_moves = [move]
            elif score == best_score:
                best_moves.append(move)

            alpha = max(alpha, best_score)

        return random.choice(best_moves)
```

3. Alpha beta pruning을 위한 준비 -alphabeta_agent.py

- Alphabotot(Agent class 상속)

- _alphabeta: alpha beta pruning을 통해 특정 수에서 얻을 수 있는 최대 점수를 구함

1) game_state: GameState class 객체, depth: 탐색 깊이, alpha/beta: alpha beta pruning을 위한 값, maximizing_player: 플레이어 본인의 차례면 True, 상대방이면 False(alpha-beta pruning 예시 설명 중 max step과 min step 판별)

2) Game이 종료된 상태: 승리 시 10만점, 무승부 시 0점, 패배 시 -10만점

3) 마지막 depth(tree에서 말단 node)인 경우: 해당 상태의 score값 반환

4) 그렇지 않을 경우 select_move에서와 동일하게 가능한 수를 탐색하고 높은 점수별로 정렬함

```
def _alphabeta(self, game_state, depth, alpha, beta, maximizing_player):
    if game_state.is_over():
        winner = game_state.winner()

        if winner == self.player:
            return 100000
        elif winner is None:
            return 0
        else:
            return -100000

    if depth == 0:
        return game_state.score(self.player)

    moves = game_state.legal_moves()
    moves = self.order_moves(game_state, moves, maximizing_player)
```

3. Alpha beta pruning을 위한 준비 -alphabeta_agent.py

- Alphetabot(Agent class 상속)
 - _alphabeta: alpha beta pruning을 통해 특정 수에서 얻을 수 있는 최대 점수를 구함
- 5) maximizing_player인 경우(max node)
 - 현재 node에서의 best_score는 -inf로 시작
 - 각 move에 대해 해당 move를 적용하고, _alphabeta를 depth를 1개 낮추어 해당 move에서의 score 구함
 - Best_score를 업데이트함
 - 이전 부모 node로부터 상속받은 alpha값보다 best_score가 클 경우 alpha값도 변경함
 - Beta값보다 alpha값이 크거나 같을 경우 alpha-beta pruning 알고리즘에 의해 추가 탐색 종료
- 6) minimizing_player인 경우(min node)
 - Min node이므로 best_score = +inf로 시작
 - Beta 값을 업데이트함
 - 이외 과정은 maximizing_player인 경우와 동일하게 진행

```
if maximizing_player:
    best_score = float("-inf")

    for move in moves:
        next_state = game_state.apply_move_fast(move)

        score = self._alphabeta(
            game_state=next_state,
            depth=depth - 1,
            alpha=alpha,
            beta=beta,
            maximizing_player=False,
        )

        best_score = max(best_score, score)
        alpha = max(alpha, best_score)

        if beta <= alpha:
            break

    return best_score

else:
    best_score = float("inf")

    for move in moves:
        next_state = game_state.apply_move_fast(move)

        score = self._alphabeta(
            game_state=next_state,
            depth=depth - 1,
            alpha=alpha,
            beta=beta,
            maximizing_player=True,
        )

        best_score = min(best_score, score)
        beta = min(beta, best_score)

        if beta <= alpha:
            break

    return best_score
```

3. Alpha beta pruning을 위한 준비 -alphabeta_agent.py

- AlphabetaAgent (Agent class 상속)
 - order_moves, move_order_score: pruning 과정을 보조하는 함수
 - order_moves 함수: 주어진 move들 중 담장의 성과가 좋은 move 순서대로 정렬하는 함수
 - Move_order_score 함수: 특정 move를 통해 얻을 수 있는 score의 이득을 계산하는 함수
- 1) target_item: move의 최종 도착지에 위치해 있는 말을 탐색 -> move_order_score가 호출되는 경우 legal_move를 통과한 move가 들어가는데, 이때 move가 도착하는 곳에 다른 말이 위치한다면 이 말은 잡히는 말임
- 2) 잡히는 말이 있다면 score에 이 말의 점수를 추가
- 3) Pawn 등이 승진하는 move라면 promotion되는 말의 점수를 추가

-> order_moves와 move_order_score의 기능: alpha-beta pruning에서 최종 node의 점수를 구하는 데에는 사용되지 않지만, 트리를 탐색하는 과정에서 단기적으로 좋은 결과를 내는 node를 먼저 탐색하게 해 alpha-beta pruning과정을 빠르게 진행하는 데 사용함

```
def order_moves(self, game_state, moves, maximizing_player):  
    return sorted(  
        moves,  
        key=lambda move: self.move_order_score(game_state, move),  
        reverse=True,  
    )  
  
def move_order_score(self, game_state, move):  
    score = 0  
  
    target_item = game_state.board.get(move.point_to)  
  
    if target_item is not None:  
        _, captured_piece = target_item  
        score += PIECE_SCORE[captured_piece]  
  
    if move.promotion is not None:  
        score += PIECE_SCORE[move.promotion]  
  
    return score
```

4. 게임 환경 제작 - player.py 업데이트

- 이전에 제작했던 player.py에 Alphatabot을 적용
 - Alphatabot 관련 import를 적용
 - Agents에서 Player를 Alphatabot으로 교체

```
from black_white.player import Player
from black_white.chessboard_for_tree import GameState
from agent_file.random_agent import Randombot
from agent_file.alphabeta_agent import Alphatabot
```

```
def main():
    game = GameState()

    agents = {
        Player.white: Alphatabot(),
        Player.black: Alphatabot(),
    }

    max_moves = 300
    move_count = 0
```

5. Agent 실행 결과

- 실행 결과: 게임 진행 양상이 이전과 동일하게 terminal에 출력될 것임.

```
Move 294: black plays Move(Point(row=6, col=6) -> Point(row=6, col=5))
7 . r b . . . . .
6 k p . . p r . p
5 . n . p . p . Q
4 . . . . . P . .
3 . . P p P . . .
2 . . . . . . . P
1 P . . . . . . .
0 . R . K . B . R

  0 1 2 3 4 5 6 7

Move 295: white plays Move(Point(row=5, col=7) -> Point(row=4, col=7))
7 . r b . . . . .
6 k p . . p r . p
5 . n . p . p . .
4 . . . . . P . Q
3 . . P p P . . .
2 . . . . . . . P
1 P . . . . . . .
0 . R . K . B . R

  0 1 2 3 4 5 6 7
```

<그림 5>
게임 진행 출력

6. 다음 시간 내용 및 개선 과제

- Alphabot의 경우 Randombot에 비해 개선된 수를 두기는 하지만, 인위적인 점수 체계에 영향을 많이 받고 경직된 수를 두는 문제가 존재할 수 있음
- Depth를 늘릴수록 탐색해야 할 move가 많아져, 수를 두는 속도가 느려짐
- 개선점: 특이 패턴 등을 저장하여 세분화된 점수 시스템을 만들기, 알고리즘 개선을 통한 속도 개선
- 다음 주: 트리 기반 탐색의 한계를 극복하기 위한 강화학습 기반 algorithm 도입
- 각 조에서 제공된 파일을 기반으로 자유롭게 개선해 나가 주시면 될 것 같습니다..!

감사합니다!